



Trabajo Práctico I: Seam Carving

Licenciatura en Tecnología Digital
Tecnología Digital V: Diseño de Algoritmos

Autores:

Timo Doeswijk
Bautista Goya

1er semestre del 2026

Índice

1. Introducción	2
1.1. Contexto	2
1.2. Objetivo	2
2. Descripción del problema	2
2.1. Ejemplo ilustrativo	3
3. Modelado	4
4. Algoritmos implementados	5
4.1. Fuerza Bruta	5
4.2. Backtracking	7
4.3. Programación Dinámica	7
4.4. Comparación entre las distintas implementaciones	9
4.5. Eficiencia	9
5. Experimentación	10
5.1. Propósito	10
5.2. Entorno de ejecución y metodología	10
6. Resultados	11
6.1. Análisis de los resultados	13
7. Conclusión	13

1. Introducción

1.1. Contexto

Dentro del campo de los programas de edición fotográfica computarizada, las herramientas a disposición al momento de modificar el tamaño de una imagen digital para alcanzar una resolución específica son acotadas y poco personalizables. Estas, siendo generalmente un recorte¹ o un escalado uniforme², comparten una dificultad: el contenido más relevante de la imagen puede sufrir distorsiones que compliquen su comprensión o, peor aún, desaparecer. Por estos motivos se introduce una nueva herramienta basada en un método completamente distinto: el **Seam Carving**.

El *Seam Carving* es una técnica de redimensionamiento consciente del contenido (*content-aware*) el cual toma trazados verticales de píxeles conectados, llamados costuras (*seams*), que atraviesan la imagen de un extremo al otro pasando por regiones de baja importancia visual. Este método preserva aquellos sectores relevantes de la imagen y sacrifica los más redundantes para alcanzar las dimensiones deseadas.

1.2. Objetivo

El presente trabajo busca dar con una aplicación efectiva del algoritmo de *Seam Carving*. Esto implica:

- plantear y explicar formalmente el problema (secciones 2 y 3);
- diseñar y proponer tres algoritmos resolutivos fundamentados del mismo: uno de Fuerza Bruta, otro de Backtracking y otro último de Programación Dinámica (sección 4);
- implementarlos en el lenguaje de programación compilado C++ para luego re-implementarlos en el lenguaje interpretado Python (sección 4);
- formular una hipótesis comparativa sobre dichos algoritmos para corroborarla mediante experimentos rigurosos (secciones 5 y 6);
- arribar a una conclusión sobre el trabajo realizado (sección 7), introducir los próximos pasos a seguir y los puntos de mejora en el trabajo.

2. Descripción del problema

Definimos como **energía** (e) a la relevancia que un píxel de una imagen representa en ella y su contexto. Dada una imagen de $n \times m$ píxeles, podemos representar la energía de cada píxel con una matriz (de tamaño $n \times m$) de valores asociados donde, para toda celda (i, j) , estas tienen un valor de energía:

$$e_{ij} \geq 0$$

Por otra parte, una **costura vertical** es una secuencia (s) de píxeles:

$$s = ((1, c_1), (2, c_2), \dots, (n, c_n))$$

¹[https://en.wikipedia.org/wiki/Cropping_\(image\)](https://en.wikipedia.org/wiki/Cropping_(image))

²https://en.wikipedia.org/wiki/Image_scaling

tal que, para todo $i \in \{1, \dots, n-1\}$, se cumple:

$$|c_{i+1} - c_i| \leq 1$$

es decir, píxeles consecutivos de la costura son adyacentes vertical o diagonalmente.

Luego, la energía de la costura se define como:

$$E(s) = \sum_{i=1}^n e_{i,c_i}$$

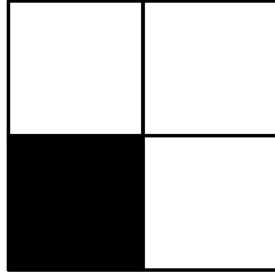
El objetivo del problema es encontrar la costura vertical de mínima energía, es decir, hallar:

$$\min_s E(s) = \min_s \sum_{i=1}^n e_{i,c_i}$$

para todas las costuras verticales de píxeles s existentes en la imagen, partiendo desde $(1, k)$, $\forall 1 \leq k \leq m$ (el píxel de la primera fila de todas las columnas).

2.1. Ejemplo ilustrativo

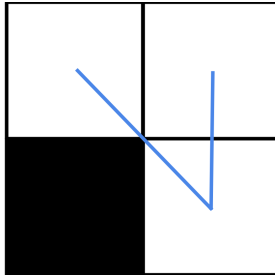
Supongamos que se tiene la matriz de energía de la siguiente imagen de 2×2 píxeles a redimensionar. Reiterando, se busca hallar las costuras verticales mínimas partiendo desde la posición $(1, k)$, $\forall k \in \mathbb{N}$, $1 \leq k \leq m$.



$$\begin{bmatrix} 1, 0 & 2, 0 \\ 8, 0 & 1, 5 \end{bmatrix}$$

Figura 1: Imagen de 2x2 pixeles y su matriz de energía asociada

Luego de aplicarle el algoritmo, este retornaría las costuras verticales:



$$s_1 = ((1, 1), (2, 2))$$

$$s_2 = ((1, 2), (2, 2))$$

Figura 2: Costuras verticales de mínima energía

viéndose así la imagen recortada luego de eliminar las costuras verticales de mínima energía en dos ocasiones.



Figura 3: Resultado luego de aplicar el algoritmo

3. Modelado

Retomando el ejemplo anterior, una posible secuencia de elecciones del algoritmo para el problema planteado podría ser:

comenzar en (1, 1) → preguntarse cuál tiene menor energía, si la posición (2, 1) o la (2, 2) → elige la (2, 2) → comenzar en (1, 2) → preguntarse cuál tiene menor energía, si la posición (2, 1) o la (2, 2) → elige la (2, 2) → fin.

Podemos diagramar cada *seam* de la salida como dos árboles de decisiones:

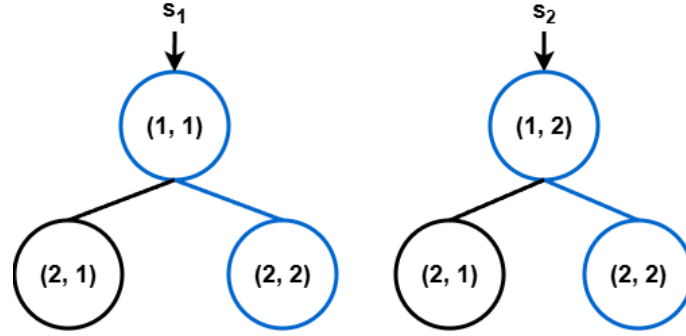


Figura 4: En celeste, las decisiones que acabaría tomando el algoritmo.

Recordamos que los píxeles del *seam* están ubicados debajo o diagonalmente adyacentes al anterior de la secuencia, por lo que, en matrices de energía con más de dos columnas y una fila, los árboles van a tener hasta tres hijos por nodo.

Con esto en cuenta, el árbol completo de decisiones generalizado para una imagen de $n \times m$ píxeles sería (con $k \in \mathbb{N}, 1 < k < m$):

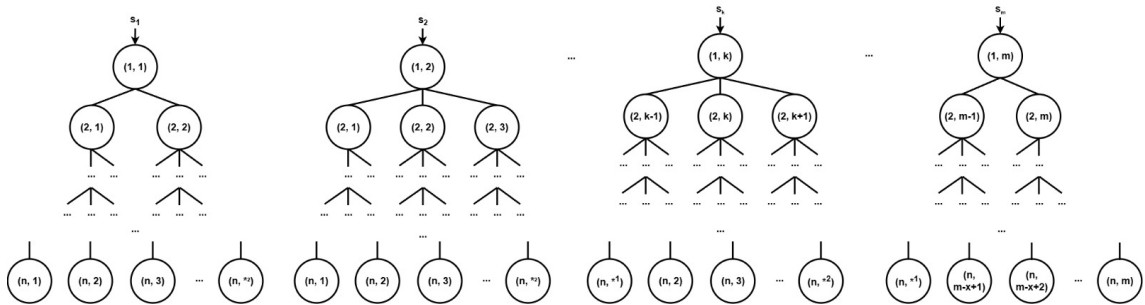


Figura 5: $*^1 = \max\{1; k - x\}$, $*^2 = \min\{m; x + k - 1\}$

Para comprender qué representa la x en el árbol generalizado, proporcionamos este ejemplo utilizando una imagen de 1200×800 píxeles donde se muestra (en celeste) **(I)** todas las posibles costuras verticales de $k = 0$, **(II)** todas las posibles costuras verticales de $k = 600$ y **(III)** todas las posibles costuras verticales de $k = m = 1200$.

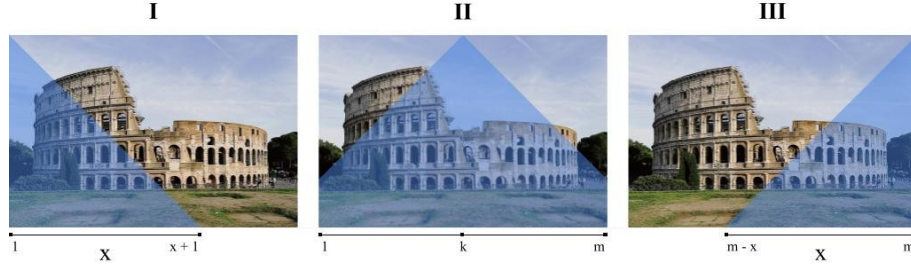


Figura 6: Ejemplos de posibles costuras verticales dada una posición de inicio

Por lo tanto, el problema se modela en cómo hallamos, mediante un algoritmo, las secuencias óptimas de decisiones en este árbol tal que, conjuntas, devuelvan el *Seam Carving* óptimo para una matriz de energía cualquiera.

4. Algoritmos implementados

4.1. Fuerza Bruta

Viendo el árbol de decisiones generalizado de la sección anterior (Figura 5), se llega rápidamente a la solución más trivial: probar todas las posibles costuras para cada columna de la matriz, quedarnos con aquella que minimiza $E(s)$ en cada caso y elegir la mínima de todas al final.

Entonces, partiendo de esta premisa, definimos a la función recursiva de Fuerza Bruta $f(i, j)$, que comienza con las celdas $(1, z)$ de una matriz de energía, con $z \in \mathbb{N}, 1 \leq z \leq m$.

$$f(i, j) = \begin{cases} e_{ij} & \text{si } i = n \\ e_{ij} + \min\{f(i+1, j), f(i+1, j+1), f(i+1, j-1)\} & \text{si } 1 < j < m \wedge 0 < i < n \\ e_{ij} + \min\{f(i+1, j), f(i+1, j+1)\} & \text{si } j = 1 \wedge 0 < i < n \\ e_{ij} + \min\{f(i+1, j), f(i+1, j-1)\} & \text{si } j = m \wedge 0 < i < n \\ 0 & \text{sino} \end{cases}$$

El caso base aparece cuando $i = n$, es decir, si la recursión alcanzó la última fila de la matriz (sin salirse del rango $n \times m$), donde simplemente devuelve la energía de la celda. Para la parte recursiva, recordamos, se busca:

$$\min_s \sum_{i=1}^n e_{i,c_i}$$

que es, en otras palabras, la suma de las energías mínimas para una costura, lo cual hace la función en sus tres casos restantes:

- si $1 < j < m \wedge 0 < i < n$, suma el mínimo entre las tres celdas adyacentes diagonal y verticalmente;
- si $j = 1 \wedge 0 < i < n$ o si $j = m \wedge 0 < i < n$, suma el mínimo entre la celda adyacente verticalmente por debajo, y una de las celdas adyacentes diagonalmente;

Para responder a la consigna construimos dos funciones: la primera es la encargada de tomar la mínima costura comenzando por cada $(1, z)$, es decir, para cada celda de la

primera fila de una matriz de energía dada; posteriormente, llama a la segunda, que es la implementación de la función f especificada anteriormente.

El problema se modeló en C++ utilizando los parámetros de la función recursiva. El primer parámetro utilizado, además de la matriz de energía, fue *sol*, que representa la costura que se está analizando en el llamado recursivo. Al llegar a la última fila, este parámetro contendrá la costura completa. Para determinar si una costura es de mínima energía, se utiliza un segundo parámetro *opt*, el cual es una variable pasada por referencia que almacena la energía mínima encontrada hasta el momento. Si la costura actual mejora este valor, se devuelve dicha costura y se actualiza el parámetro. En caso contrario, se retorna un vector vacío. Finalmente, se selecciona el último vector no vacío, el cual corresponderá a la costura óptima de las analizadas.

Se considera que esta implementación corresponde a una aplicación válida del algoritmo de *Fuerza Bruta*, ya que se analiza la totalidad de las combinaciones posibles hasta agotar el espacio de búsqueda.

El siguiente pseudocódigo representa ambas implementaciones, que recibe e como la matriz de tamaño $n \times m$ (vector de vectores) y devuelve un vector $\{c_1, \dots, c_n\}$ tal que la costura óptima $s = ((1, c_1), (2, c_2), \dots, (n, c_n))$.

```
FuerzaBruta(e)
  if e = vacio then
    return vacio
  else
    seamOpt <- vacio
    energiaOpt <- -1
    for i < m do
      seamLoc <- encontrarSeam(e, {i}, energiaOpt)
      if seamLoc no está vacío then
        seamOpt <- seamLoc
      end if
    end for
    return seamOpt
  end if

encontrarSeam(e, sol, opt):
  if j < 1 or j > m then
    return vacio // C.B. I: Posición inválida
  else if i = n then // C. B. II: ltima fila
    if e[i][j] < opt then
      opt <- e[i][j]
      return sol
    return vacio
  else
    // Casos recursivos
    sol' <- sol + [j-1]
    s1 <- encontrarSeam(e, sol', opt)
    sol' <- sol + [j]
    s2 <- encontrarSeam(e, sol', opt)
    sol' <- sol + [j+1]
    s3 <- encontrarSeam(e, sol', opt)

    if s3 no es vacio then return s3
```

```

else if s2 no es vacio then return s2
else if s1 no es vacio then return s1
end if
return vacio
end if

```

4.2. Backtracking

A partir del enfoque de Fuerza Bruta, se implementó una poda sobre la función recursiva para obtener un algoritmo de *Backtracking*. Durante la ejecución, si la energía acumulada de la costura parcial supera el valor óptimo actual (la mejor costura global encontrada hasta el momento), se descarta dicha rama de ejecución y se retorna un vector vacío. Esto evita explorar caminos subóptimos que no conducirán a una mejor solución, fundamentado en la propiedad de que los valores de energía en la matriz son no negativos ($e_{ij} \geq 0$).

Con esto en claro, la única modificación al pseudocódigo anterior fue la siguiente:

```

def encontrarSeam'(e, sol, opt):
    ...
    if energia(e, sol) > opt then
        return vacío
    ...

```

▲ *energia(e, sol)* es la función que calcula la energía de la costura parcial *sol*.

4.3. Programación Dinámica

En los códigos de Fuerza Bruta y Backtracking, las posibles costuras de las distintas columnas se superponen entre sí y, como los algoritmos las prueban todas, se producen repeticiones de estado constantemente.



Figura 7: Si combinamos los tres ejemplos de la Figura 6, se pueden ver las repeticiones.

Para mitigar esto, usamos Programación Dinámica y el acercamiento top-down para aprovechar el código de las funciones recursivas ya programadas.

En un inicio, buscamos construir una matriz de vectores para almacenar las respuestas de cada llamado recursivo (tabla de memoización), lo cual creaba un problema de memoria debido a que los vectores ocupaban demasiado espacio. La solución fue construir una

matriz de energías mínimas (de `doubles`, más pequeña en memoria) para que luego se reconstruya la costura óptima a partir de ella.

Entonces, dada la matriz de energía e de tamaño $n \times m$ y una tabla de memoización inicializada con -1 :

```
ProgramacionDinamica(e):
  if e esta vacio then
    return vacio
  else
    memoizacion <- {-1}^{n x m}
    minCol <- 1
    for j <= m do
      encontrarSeam''(e, memoizacion, 1, j)
      if memoizacion[1][j] < memoizacion[1][minCol] then
        minCol <- j
      end if
    end for
    seamOpt <- vacio
    seamOpt[1] <- minCol
    for i < n + 1 do
      col <- seamOpt[i-1]
      energiaOpt <- col
      if col > 0 and memoizacion[i][col-1] < memoizacion[i][
energiaOpt] then
        energiaOpt <- col - 1
      end if
      if col < m - 1 and memoizacion[i][col+1] < memoizacion[i][
energiaOpt] then
        energiaOpt <- col + 1
      end if
      seamOpt[i] <- energiaOpt
    end for
    return seamOpt
  end if

encontrarSeam''(e, memoizacion, i, j):
  if j < 1 or j > m then // C.B. I: Posición inválida
    return infinito
  else if memoizacion[i][j] != -1 then // Caso P.D.
    return memoizacion[i][j]
  else if i = n then // C. B. II: ltima fila
    memoizacion[i][j] = e[i][j]
    return memoizacion[i][j]
  else
    // Casos recursivos
    s1 <- encontrarSeam''(e, memoizacion, i + 1, j - 1)
    s2 <- encontrarSeam''(e, memoizacion, i + 1, j)
    s3 <- encontrarSeam''(e, memoizacion, i + 1, j + 1)
    memoizacion[i][j] = e[i][j] + min(s1, s2, s3)
    return memoizacion[i][j]
  end if
```

4.4. Comparación entre las distintas implementaciones

La implementación del algoritmo de Fuerza Bruta se desarrolló de manera directa siguiendo el diseño conceptual previo; no obstante, su codificación en C++ presentó desafíos en la gestión de estructuras de datos para representar el espacio de soluciones de forma eficiente. Por el contrario, la transición hacia un esquema de Backtracking resultó altamente eficiente en términos de desarrollo. Al basarse en la lógica de Fuerza Bruta, la adaptación consistió simplemente en la integración de una condición de poda, con cambios mínimos en el código fuente. Finalmente, la técnica de Programación Dinámica representó el mayor desafío de implementación, ya que a diferencia de los enfoques anteriores, esta metodología requirió una reestructuración de la lógica para el llenado de la matriz de costos mínimos. Una vez superado este cambio de paradigma, la codificación final se consolidó de manera estable.

Todos los algoritmos anteriores se tradujeron al lenguaje Python para poder comparar sintaxis y, más adelante en este informe, tiempos de procesamiento. Con el fin de asegurar una comparativa objetiva entre lenguajes, se decidió replicar las implementaciones previas de C++ sin introducir mejoras de optimización. Este requerimiento de paridad fue comunicado y supervisado durante la interacción con las herramientas de Inteligencia Artificial utilizadas. El proceso resultó en un código fuente que satisface los requisitos de equivalencia algorítmica necesarios para la comparación posterior.

La herramienta utilizada fue **GitHub Copilot de VS Code**, integrando el modelo **Claude Haiku 4.5**.³

Para la utilización de los algoritmos, se debe ejecutar cada archivo por separado con los mismo parámetros que en C++. Es requerida la instalación de la librería PIL/Pillow (con el comando *pip install Pillow*) para ejecutar el archivo en modo imagen.

4.5. Eficiencia

La siguiente tabla expone sobre la complejidad temporal de los tres algoritmos implementados:

Tipo	Fuerza Bruta	Backtracking	Programación Dinámica
Principal	$O(m)$	$O(m)$	$O(1)$
Auxiliar	$O(3^n)$	$O(3^n)$ (peor caso)	$O(m \times n)$
Total	$O(m \times 3^n)$	$O(m \times 3^n)$ (peor caso)	$O(m \times n)$
Justificación	Los tres llamados recursivos por iteración calculan todas las posibles costuras, los cuales se realizan m veces.	En el peor caso, donde la mejor costura está en la última columna, el algoritmo no va a poder realizar ninguna poda.	La energía óptima de cada celda se calcula una vez y se consulta la tabla ($O(1)$). Resulta en $O(m \times n \times 1)$.

El siguiente gráfico muestra cómo evoluciona el tiempo de cómputo al que crecer n para cada algoritmo con $m = 10$:

³Link a la conversación (no pudo ser copiado el enlace directo): https://docs.google.com/document/d/1p0VnRPL9ri0hNmsgsRf58fUIC_TRL7UA2ETht9ntYctI/edit?usp=sharing

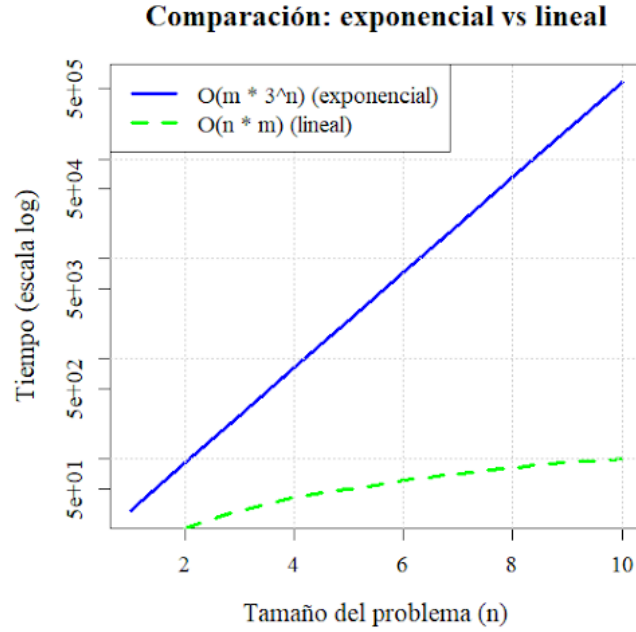


Figura 8: Comparación: exponencial vs lineal

Se observa como, en la teoría, el crecimiento de los tiempos de los algoritmos de Fuerza Bruta y Backtracking no se detiene, mientras que el de Programación Dinámica lo hace decrecientemente para una matriz de como máximo 10×10 . Más adelante veremos cómo estos dos primeros algoritmos son imprácticos de utilizar para imágenes de tamaños más realistas.

5. Experimentación

5.1. Propósito

La fase experimental descrita a continuación tiene como propósito validar de manera empírica las hipótesis planteadas sobre la complejidad temporal de los algoritmos desarrollados. Asimismo, se busca contrastar el desempeño relativo entre los lenguajes de programación seleccionados, analizando si los resultados observados guardan correlación con el comportamiento teórico esperado:

1. El algoritmo de Backtracking mejora sutilmente el tiempo de ejecución con respecto al algoritmo de Fuerza Bruta.
2. La implementación de Programación Dinámica mejora ampliamente el tiempo de ejecución con respecto al resto de algoritmos implementados anteriormente.
3. A pesar de la mayor complejidad sintáctica que presenta C++ en comparación con Python, el primero se posiciona como la opción preferible para este desarrollo, debido a su capacidad para realizar una gestión de memoria de bajo nivel y una ejecución compilada, factores críticos para minimizar el overhead en algoritmos de alta demanda computacional.

5.2. Entorno de ejecución y metodología

El entorno de Hardware y Software utilizado para la experimentación es el siguiente:

- CPU: Intel Core i5-1335U
- Memoria RAM: 16GB SODIMM a 3200 MT/s
- Sistema Operativo: Ubuntu 24.04.4 LTS ejecutado sobre Windows Subsystem for Linux (WSL 2).
- Compilador: C++ con GCC 15.2.0.
- Herramienta de medición: Las mediciones de tiempo se automatizaron utilizando *hyperfine*, configurado para realizar 5 ejecuciones de calentamiento y 10 mediciones por cada instancia para promediar los segundos exactos que el procesador pasó ejecutando exclusivamente las instrucciones del programa.

Durante los experimentos, se midió la esperanza, la desviación estandar, el mínimo, el máximo y la cantidad de outliers de los *tiempos totales de cómputo* en los peores casos en matrices de cada tamaño. El peor caso ocurre cuando la costura de mínima energía óptima es la última analizada. A partir de nuestra implementación en código, la última costura vertical analizada es la última columna de la matriz de energía. Los datos tienen distribución uniforme con un rango entre 7.0 y 9.0 para las columnas no optimas, y de entre 0.1 y 0.3 en la columnas optima. Las matrices utilizadas son:

Pequeña (7×7)	Mediana (10×10)	Grande (15×15)
<pre>[7.2 8.7 7.8 7.8 7.2 9.5 1.9] [9.3 7.1 7.4 8.6 9.4 8.9 1.9] [8.8 9.8 7.9 9.9 7.6 9.0 1.2] [7.3 9.8 8.1 8.5 9.7 9.4 2.4] [8.8 7.3 7.9 7.7 7.2 8.9 0.5] [7.3 9.3 8.5 9.0 8.2 9.0 1.7] [9.9 8.6 9.5 9.7 9.9 9.0 2.2]</pre>	<pre>[9.7 8.3 7.3 9.2 7.3 8.1 9.7 8.3 7.7 1.0] [9.4 9.6 7.7 8.2 9.0 8.0 9.8 9.8 7.7 2.0] [7.3 7.3 7.9 8.2 9.6 8.6 8.3 8.5 7.2 1.4] [7.1 8.2 9.1 9.0 9.4 7.9 8.6 8.8 7.2 1.4] [8.0 8.7 9.0 9.5 9.4 9.2 8.2 9.2 8.6 1.6] [8.4 7.4 9.6 9.1 8.8 8.2 9.3 8.6 8.7 2.4] [8.1 7.1 7.5 7.8 7.8 9.3 7.5 7.9 7.8 2.1] [8.3 7.8 8.5 9.5 9.0 8.4 9.4 7.1 7.1 2.5] [8.4 7.2 8.3 8.9 9.5 7.4 8.8 9.0 8.3 0.2] [8.8 8.6 8.2 9.8 8.6 8.0 9.0 8.5 7.7 0.6]</pre>	<pre>[9.8 8.8 8.8 8.1 9.8 8.2 9.2 8.1 7.2 9.2 7.9 9.6 8.9 8.6 2.7] [9.9 8.6 8.9 7.0 8.0 7.1 9.3 7.3 8.2 7.3 9.3 8.0 9.4 8.2 2.1] [7.8 9.6 8.8 7.4 9.0 9.6 8.4 8.4 8.5 8.8 8.9 7.9 8.9 7.6 1.7] [8.1 8.1 9.0 7.1 8.3 8.6 7.8 9.5 9.7 9.4 9.4 7.3 8.7 8.0 0.1] [7.4 7.4 8.0 8.7 7.1 9.8 8.9 9.5 9.6 9.9 9.3 7.7 9.1 7.9 2.5] [9.4 8.8 9.6 8.2 7.2 7.2 7.8 8.0 9.4 9.6 7.4 8.7 7.8 7.6 1.1] [7.8 9.5 9.1 8.3 7.9 7.9 9.0 7.6 7.7 9.0 8.7 8.0 8.6 8.3 1.1] [7.1 9.6 9.1 9.5 8.4 7.0 8.8 7.9 8.4 7.8 8.8 7.2 8.6 8.1 2.0] [8.5 8.3 7.5 8.6 8.1 9.6 7.8 7.9 7.9 9.6 9.7 7.5 8.3 9.3 2.8] [7.4 9.5 8.3 9.8 7.5 8.1 8.7 8.8 7.7 8.4 7.0 7.0 9.7 9.4 3.0] [8.6 9.4 7.8 8.5 7.1 8.1 8.3 8.3 7.9 7.5 8.4 8.1 7.0 8.0 0.6] [8.8 7.9 8.4 8.7 9.6 8.4 8.9 9.1 8.4 8.4 9.4 9.8 7.3 7.5 0.6] [8.6 8.1 8.4 9.2 7.3 9.4 7.5 8.4 7.2 8.8 9.1 7.2 9.5 7.7 1.8] [7.7 8.1 9.4 9.5 7.1 9.2 9.0 7.9 9.3 8.1 9.1 8.2 7.9 8.9 2.7] [8.3 9.4 8.9 8.0 7.3 8.8 9.1 9.5 8.4 7.7 7.2 8.0 8.7 9.0 1.3]</pre>

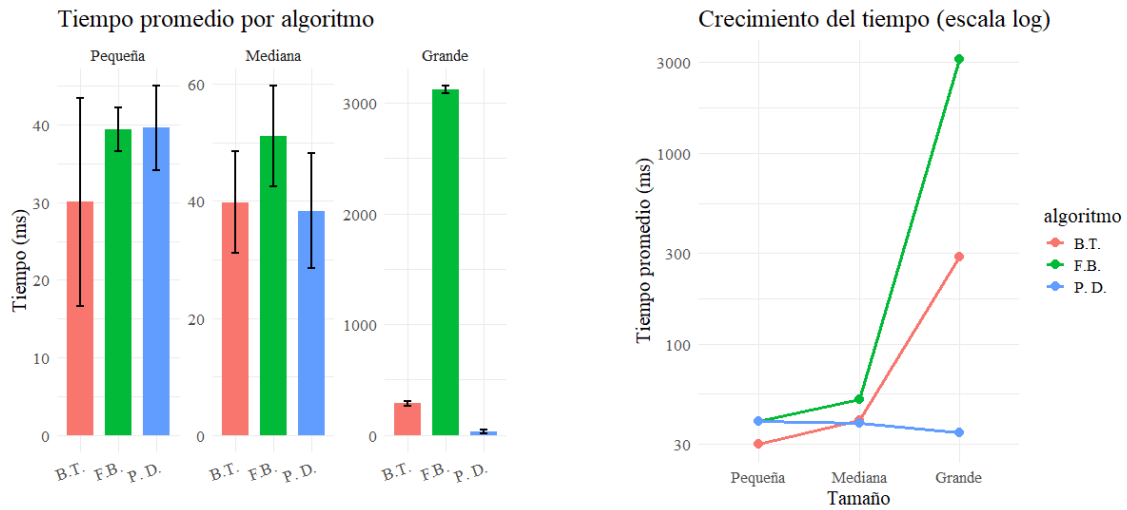
Cuadro 1: Matrices de peor caso aleatorias de tamaños 7×7 , 10×10 y 15×15 .

6. Resultados

Para el lenguaje C++, los resultados obtenidos fueron los siguientes:

Algoritmo	Tamaño	Esperanza (μ)	Desviación (σ)	Mínimo	Máximo
Fuerza Bruta	Pequeña	39.415	2.840	34.651	42.744
Backtracking	Pequeña	30.077	13.405	19.489	51.782
Prog. Dinámica	Pequeña	39.578	5.452	31.366	47.789
Fuerza Bruta	Mediana	51.121	8.581	36.907	64.264
Backtracking	Mediana	39.774	8.702	27.149	57.222
Prog. Dinámica	Mediana	38.332	9.825	20.041	49.610
Fuerza Bruta	Grande	3118.828	35.426	3082.896	3181.591
Backtracking	Grande	286.642	21.068	264.255	338.833
Prog. Dinámica	Grande	34.307	16.447	9.504	51.450

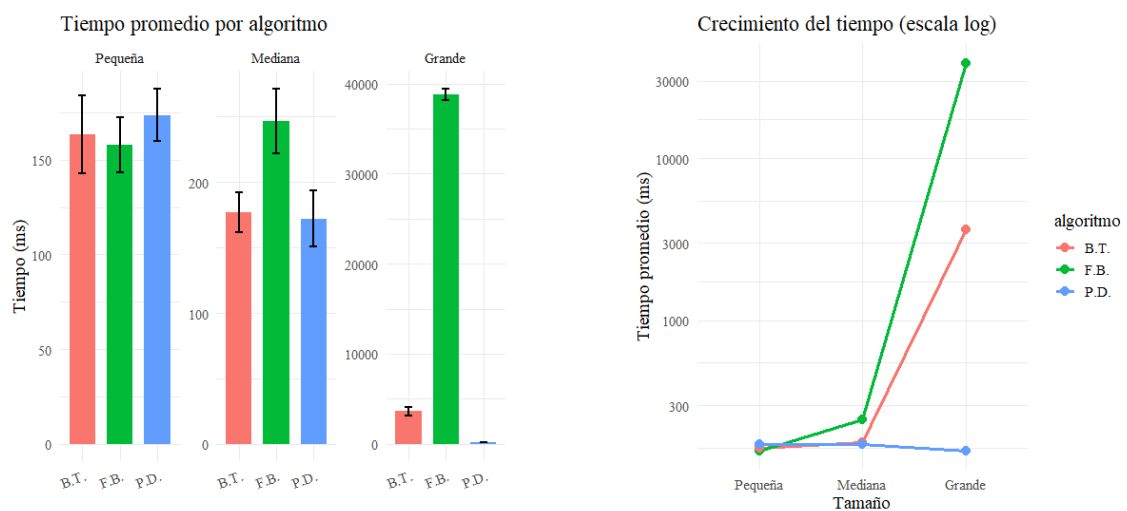
Cuadro 2: Análisis estadístico de los tiempos de cómputo totales en C++.



Por otra parte, para el lenguaje Python, obtuvimos los siguientes resultados:

Algoritmo	Tamaño	Esperanza (μ)	Desviación (σ)	Mínimo	Máximo
Fuerza Bruta	Pequeña	157.830	14.425	129.941	176.701
Backtracking	Pequeña	163.294	20.622	111.560	188.504
Prog. Dinámica	Pequeña	173.721	13.994	145.469	198.264
Fuerza Bruta	Mediana	246.676	24.902	218.852	291.896
Backtracking	Mediana	176.769	15.161	153.700	203.948
Prog. Dinámica	Mediana	172.188	21.626	122.868	196.769
Fuerza Bruta	Grande	38825.306	653.430	38030.206	39732.024
Backtracking	Grande	3621.361	477.039	3190.096	4394.628
Prog. Dinámica	Grande	157.673	23.876	126.515	202.861

Cuadro 3: Análisis estadístico de los tiempos de cómputo totales en Python.



Se ha optado por presentar los resultados del experimento mediante una combinación de tablas y gráficos ya que las primeras facilita un análisis y comparación directa del rendimiento entre los lenguajes de programación evaluados, mientras que los segundos permite una visualización intuitiva de las disparidades de eficiencia entre los distintos algoritmos implementados.

6.1. Análisis de los resultados

En primer lugar, la hipótesis que postulaba una mejora sutil del algoritmo de Backtracking frente a la Fuerza Bruta debe ser parcialmente refutada, ya que la optimización práctica resulta ser mayor a la que esperábamos. Si bien en instancias pequeñas la diferencia es mínima, e incluso perjudicial en Python, al escalar a matrices grandes el Backtracking logra reducir el tiempo de cómputo en aproximadamente un 90 % en ambos lenguajes. Esto demuestra que las podas implementadas en el árbol de recursión logran descartar ramas de cálculo de manera eficiente ante el crecimiento de la entrada.

Respecto a la segunda hipótesis, la superioridad de la Programación Dinámica se confirma para instancias de gran tamaño, aunque revela un comportamiento inesperado en matrices más reducidas. En los tamaños grandes, este enfoque domina de forma absoluta dado que su complejidad no es exponencial, resolviendo el problema en fracciones de segundo frente al colapso temporal de los otros métodos. Sin embargo, en instancias pequeñas, la Programación Dinámica no es el algoritmo más rápido, siendo superado en eficiencia por el Backtracking y compitiendo a la par con la Fuerza Bruta. Este patrón inesperado revela que el overhead asociado a la inicialización de estructuras de datos auxiliares y la reserva de memoria para la tabla de doble entrada penaliza el rendimiento.

Por otro lado, la tercera hipótesis se confirma en su totalidad, consolidando a C++ como la opción preferible frente a Python para este dominio. Los datos muestran de forma consistente la superioridad del lenguaje compilado frente al interpretado; C++ llega a ser más de 12 veces más rápido frente al peor escenario de Python, y unas 4.5 veces más rápido al comparar las implementaciones óptimas. La naturaleza interpretada de Python y la gestión delegada de la memoria, introduce una sobrecarga crítica y constante. En contraparte, la gestión de memoria de bajo nivel de C++ resulta ser un factor indispensable para este tipo de algoritmos con alta exigencia computacional.

Los resultados demuestran que, si bien las diferencias en el tiempo de ejecución son marginales para instancias de entrada pequeñas, la implementación mediante Programación Dinámica ofrece escalabilidad superior al procesar matrices de energía de gran tamaño. Esta optimización en la complejidad temporal se acentúa al utilizar C++ como lenguaje. En consecuencia, se recomienda la adopción del paradigma de **Programación Dinámica** bajo el ecosistema de C++, garantizando así una eficiencia operativa óptima en la práctica.

7. Conclusión

En este trabajo se logró alcanzar tres implementaciones funcionales, en dos lenguajes de programación cada uno, de un algoritmo de Seam Carving, siendo la tercera, la de Programación Dinámica, y aquellas en C++ las más efectivas temporalmente y, por lo tanto, las más aplicables a casos de mayor complejidad.

Aún así, los exámenes se vieron limitados, por su enfoque intencionalmente estadístico

y no práctico, primero a únicamente los peores casos, y segundo a matrices predefinidas que pueden o no representar imágenes reales. Por ende, el paso siguiente a este trabajo recae en repetir la etapa experimental para el caso de imágenes más afines a la cotidianidad (que no siempre son el peor caso ni mucho menos matrices de números). Aquí se revela una nueva rama de análisis, donde se puede estudiar cómo se comporta el algoritmo frente a un elemento que no es únicamente numérico, sino una pieza visual, con contexto propio e independiente al de cualquier otra.

Reconocemos que el código fuente podría optimizarse aún más mediante el uso de herramientas externas, tales como las de inteligencia artificial generativa; sin embargo, se tomó la decisión deliberada de restringir el alcance de estas tecnologías, evitando que refactorizaran nuestra lógica. Esta limitación voluntaria se fundamenta en la necesidad de sostener el propósito didáctico del trabajo práctico: delegar la optimización estructural a un modelo habría opacado el objetivo central, que es asimilar de manera genuina y directa el diseño, el comportamiento y la complejidad real de los paradigmas algorítmicos abordados.